ЗВІТ ДО ЛАБОРАТОРНОЇ РОБОТИ №1

Дисципліна: Функційне програмування
Варіант 11: Мобільний тариф
Студент(ка): _Магора Максим_ Група: __ФеП-23__

1. Тема та мета

Основи функціонального програмування у Python. Мета: засвоїти чисті функції, референтну прозорість, відокремлення I/O, параметризацію правил через typing.Callable.

2. Постановка завдання

За даними про мобільний номер, тариф, хвилини, SMS та трафік обчислити місячну вартість послуг. Політики тарифікації, бонусів і фільтрації реалізувати через Callable; результат записувати до нового поля monthly_cost, не змінюючи початкові дані.

3. Математична модель

C = B + max(0, M − Lm)·Pm + max(0, S − Ls)·Ps + max(0, D − Ld)·Pd; monthly_cost = max(0, bonus(C)). Тут B — абонплата, M/S/D — фактичні хвилини/SMS/ГБ, L — ліміти, P — ціни перевищення. Для демонстрації bonus(C) = 0,9·C.

4. Структура проєкту

lab01/
  README.md
  core.py
  app.py
  requirements.txt
  tests/test_core.py

5. Програмна реалізація

core.py

```
"""Pure mobile tariff calculations for laboratory work 1, variant 11."""
from typing import Callable, TypedDict


class MobileUsage(TypedDict):
    mobile: str
    tariff: str
    minutes: int
    sms: int
    traffic: float  # GB
```

```
class MobileResult(MobileUsage):
    monthly_cost: float


TariffFn = Callable[[MobileUsage], float]
BonusFn = Callable[[float], float]
FilterFn = Callable[[MobileUsage], bool]


def calculate_minutes_cost(minutes: int, limit: int, price: float) -> float:
    return max(0, minutes - limit) * price


def calculate_sms_cost(sms: int, limit: int, price: float) -> float:
    return max(0, sms - limit) * price


def calculate_traffic_cost(traffic: float, data_limit: float, price: float) -> float:
    return max(0.0, traffic - data_limit) * price


def calculate_monthly_cost(usage: MobileUsage, *, base_cost: float,
                           minutes_limit: int, minutes_price: float,
                           sms_limit: int, sms_price: float,
                           data_limit: float, traffic_price: float) -> float:
    return round(base_cost
                 + calculate_minutes_cost(usage['minutes'], minutes_limit, minutes_price)
                 + calculate_sms_cost(usage['sms'], sms_limit, sms_price)
                 + calculate_traffic_cost(usage['traffic'], data_limit, traffic_price), 2)


def with_monthly_cost(usage: MobileUsage, monthly_cost: float) -> MobileResult:
    return {**usage, 'monthly_cost': round(monthly_cost, 2)}


def filter_tariffs(usages: list[MobileUsage], accept: FilterFn) -> list[MobileUsage]:
    return [dict(usage) for usage in usages if accept(usage)]


def make_processor(*, accept: FilterFn, tariff_policy: TariffFn,
                   bonus_policy: BonusFn) -> Callable[[list[MobileUsage]], list[MobileResult]]:
    def process(usages: list[MobileUsage]) -> list[MobileResult]:
        return [with_monthly_cost(usage, max(0.0, bonus_policy(tariff_policy(usage))))
                for usage in usages if accept(usage)]
    return process
```

app.py

```
"""I/O shell: only this module prints results."""
from core import MobileResult, MobileUsage, calculate_monthly_cost, make_processor


def render_report(results: list[MobileResult]) -> None:
    for row in results:
        print(f"Номер: {row['mobile']}; тариф: {row['tariff']}; "
```

```
            f"місячна вартість: {row['monthly_cost']:.2f} грн")


def main() -> None:
    usages: list[MobileUsage] = [
        {'mobile': '0501112233', 'tariff': 'Standard', 'minutes': 120, 'sms': 40, 'traffic':
12.0},
        {'mobile': '0672223344', 'tariff': 'Standard', 'minutes': 80, 'sms': 70, 'traffic': 8.0},
        {'mobile': '0933334455', 'tariff': 'Archive', 'minutes': 40, 'sms': 10, 'traffic': 3.0},
    ]
    tariff_policy = lambda usage: calculate_monthly_cost(
        usage, base_cost=150.0, minutes_limit=100, minutes_price=1.5,
        sms_limit=50, sms_price=1.0, data_limit=10.0, traffic_price=20.0)
    bonus_policy = lambda cost: cost * 0.9  # 10% bonus discount
    accept = lambda usage: usage['tariff'] == 'Standard'
    processor = make_processor(accept=accept, tariff_policy=tariff_policy,
                               bonus_policy=bonus_policy)
    render_report(processor(usages))



if __name__ == '__main__':
    main()
```

tests/test_core.py

```
from copy import deepcopy

from core import (calculate_minutes_cost, calculate_sms_cost,
                  calculate_traffic_cost, calculate_monthly_cost,
                  filter_tariffs, make_processor)



def sample():
    return [{'mobile': '0501112233', 'tariff': 'Standard', 'minutes': 120,
             'sms': 40, 'traffic': 12.0}]



def policy(usage):
    return calculate_monthly_cost(usage, base_cost=150, minutes_limit=100,
                                  minutes_price=1.5, sms_limit=50, sms_price=1,
                                  data_limit=10, traffic_price=20)



def processor():
    return make_processor(accept=lambda u: u['tariff'] == 'Standard',
                          tariff_policy=policy, bonus_policy=lambda cost: cost * 0.9)



def test_calculations():
    assert calculate_minutes_cost(120, 100, 1.5) == 30
    assert calculate_sms_cost(40, 50, 1) == 0
    assert calculate_traffic_cost(12, 10, 20) == 40
    assert policy(sample()[0]) == 220



def test_referential_transparency():
    usages = sample()
    assert processor()(usages) == processor()(usages)
```

```
def test_no_mutation():
    usages = sample()
    before = deepcopy(usages)
    processor()(usages)
    assert usages == before
    assert 'monthly_cost' not in usages[0]


def test_callable_policies():
    result = processor()(sample())
    assert result[0]['monthly_cost'] == 198
    assert result[0]['tariff'] == 'Standard'


def test_filter_tariffs():
    usages = sample() + [{'mobile': '1', 'tariff': 'Archive', 'minutes': 0,
                          'sms': 0, 'traffic': 0.0}]
    filtered = filter_tariffs(usages, lambda u: u['tariff'] == 'Standard')
    assert len(filtered) == 1
    assert filtered[0] is not usages[0]
```

requirements.txt

```
pytest>=8.0
mypy>=1.10
black>=24.0
ruff>=0.6
```

6. Результати тестування

Команда: python -m pytest -q
Результат: 5 passed. Перевірено формули, референтну прозорість, незмінність вхідних даних, Callable-політики та фільтрацію.

Контрольний приклад: 120 хв, 40 SMS, 12 ГБ → (150 + 30 + 0 + 40) × 0,9 = 198,00 грн. Для другого запису: (150 + 0 + 20 + 0) × 0,9 = 153,00 грн.

7. Висновки

Реалізовано чисте розрахункове ядро мобільного тарифу з явною передачею параметрів, без мутації вхідних даних. Фабрика make_processor приймає політики фільтрації, тарифікації та бонусу як Callable. Побічні ефекти ізольовано в app.py. Усі п’ять автоматизованих тестів пройдено.